

C++ ARRAYS & STRINGS

Placement + MCQ + Logic Building Notes

PART A — ARRAYS IN C++

1. What is an Array?

An **array** is a collection of elements of the **same data type**, stored in **contiguous memory locations**.

Instead of creating:

```
int m1 = 10;  
int m2 = 20;  
int m3 = 30;  
int m4 = 40;  
int m5 = 50;
```

we can write:

```
int marks[5] = {10, 20, 30, 40, 50};
```

Now **marks** represents five integer values.

Why do we need arrays?

Suppose we have marks of 100 students.

Without an array:

```
marks1  
marks2  
marks3  
...  
marks100
```

With an array:

```
int marks[100];
```

The major advantages are:

- Easy storage of multiple values
 - Easy traversal
 - Easy searching
 - Easy sorting
 - Easy manipulation
 - Efficient memory organisation
 - Foundation for many DSA structures
-

2. Array Declaration

Syntax

```
data_type array_name[size];
```

Example:

```
int marks[5];
```

This creates an array capable of storing 5 integers.

Other examples:

```
float price[10];
```

```
char letters[26];
```

```
double salary[20];
```

3. Array Initialization

Method 1 — Complete initialization

```
int a[5] = {10, 20, 30, 40, 50};
```

Memory conceptually:

Index:	0	1	2	3	4
Value:	10	20	30	40	50

Accessing elements

```
cout << a[0];
```

Output:

```
10
cout << a[3];
```

Output:

```
40
```

Remember:

Array indexing starts from 0.

4. Partial Initialization

Consider:

```
int a[5] = {10, 20};
```

The remaining elements are initialized to zero.

Conceptually:

```
10 20 0 0 0
```

Example:

```
#include <iostream>
using namespace std;

int main()
{
    int a[5] = {10, 20};

    for(int i = 0; i < 5; i++)
        cout << a[i] << " ";
}
```

Output:

```
10 20 0 0 0
```

MCQ point

```
int a[5] = {};
```

All elements become:

```
0 0 0 0 0
```

5. Size Can Be Inferred

You can write:

```
int a[] = {10,20,30,40};
```

C++ automatically determines the size.

Therefore:

Size = 4

Example:

```
cout << sizeof(a) / sizeof(a[0]);
```

Output:

4

Important

This technique works when `a` is actually an array in that scope.

```
sizeof(a) / sizeof(a[0])
```

is a very common MCQ/interview question.

6. Array Indexing

Given:

```
int a[] = {10,20,30,40,50};
```

The indexes are:

Value: 10 20 30 40 50

Index: 0 1 2 3 4

Therefore:

`a[0]` → 10

`a[1]` → 20

a[2] → 30
a[3] → 40
a[4] → 50

Important

a[5]

is **outside the valid range**.

Valid indexes:

0 to size-1

C++ does not automatically perform bounds checking for built-in arrays. Reading or writing outside the valid range leads to **undefined behavior**.

7. Updating an Array Element

Arrays are mutable.

```
int a[5] = {10,20,30,40,50};
```

```
a[2] = 100;
```

```
cout << a[2];
```

Output:

100

The array becomes:

10 20 100 40 50

8. Traversing an Array

Traversal means:

Visiting every element of the array.

Example:

```
int a[5] = {10,20,30,40,50};
```

```
for(int i=0; i<5; i++)
{
    cout << a[i] << " ";
}
```

Output:

10 20 30 40 50

General pattern

```
for(int i = 0; i < n; i++)
{
    // process a[i]
}
```

Students should recognise this as the **fundamental array pattern**.

9. Reverse Traversal

```
int a[5] = {10,20,30,40,50};
```

```
for(int i=4; i>=0; i--)
{
    cout << a[i] << " ";
}
```

Output:

50 40 30 20 10

More generally:

```
for(int i=n-1; i>=0; i--)
```

10. Taking Array Input

```
int n;
cin >> n;
```

```
int a[100];
```

```
for(int i=0; i<n; i++)
```

```
{  
    cin >> a[i];  
}
```

Example input:

```
5  
10 20 30 40 50
```

The array contains:

```
10 20 30 40 50
```

Placement habit

Teach students to separate:

Input

Processing

Output

This makes debugging much easier.

11. Sum of Array Elements

Logic

Start with:

```
int sum = 0;
```

Then add every element.

```
int a[] = {10,20,30,40,50};  
int sum = 0;
```

```
for(int i=0; i<5; i++)  
{  
    sum = sum + a[i];  
}
```

```
cout << sum;
```

Output:

```
150
```

Dry run

sum = 0

10 → 10

20 → 30

30 → 60

40 → 100

50 → 150

This is an important pattern:

Accumulator pattern

12. Find Maximum Element

```
int a[] = {25,10,80,45,30};
```

```
int max = a[0];
```

```
for(int i=1; i<5; i++)  
{  
    if(a[i] > max)  
        max = a[i];  
}
```

```
cout << max;
```

Output:

80

Why start with **a[0]**?

Because we need an actual array value as our initial candidate.

Avoid blindly doing:

```
int max = 0;
```

because it fails for arrays containing only negative values.

Example:

-10 -20 -5 -30

Correct answer:

13. Find Minimum Element

Same logic.

```
int a[] = {25,10,80,45,30};
```

```
int min = a[0];
```

```
for(int i=1; i<5; i++)  
{  
    if(a[i] < min)  
        min = a[i];  
}
```

```
cout << min;
```

Output:

10

14. Count Even and Odd Numbers

```
int a[] = {10,15,22,31,40};
```

```
int even = 0;
```

```
int odd = 0;
```

```
for(int i=0; i<5; i++)  
{  
    if(a[i] % 2 == 0)  
        even++;  
    else  
        odd++;  
}
```

```
cout << "Even = " << even << endl;
```

```
cout << "Odd = " << odd;
```

Output:

Even = 3

Odd = 2

Pattern:

Condition → Counter

This becomes very useful in placement problems.

15. Linear Search

Suppose:

10 25 30 45 60

We want to find:

45

We check elements one by one.

```
int a[] = {10,25,30,45,60};
```

```
int key = 45;
```

```
for(int i=0; i<5; i++)  
{  
    if(a[i] == key)  
    {  
        cout << "Found at index " << i;  
        break;  
    }  
}
```

Output:

Found at index 3

Complexity

Best case : $O(1)$

Worst case : $O(n)$

16. Reverse an Array Using Two Pointers

This is an important **placement pattern**.

Suppose:

10 20 30 40 50

Use:

left → ← right

Initially:

left = 0

right = 4

Swap:

10 ↔ 50

Then:

20 ↔ 40

Code:

```
int a[] = {10,20,30,40,50};
```

```
int left = 0;
```

```
int right = 4;
```

```
while(left < right)
```

```
{
```

```
    swap(a[left], a[right]);
```

```
    left++;
```

```
    right--;
```

```
}
```

Result:

50 40 30 20 10

Important pattern

Two Pointers

Students will meet this pattern repeatedly in DSA.

17. Second Largest Element

Consider:

10 50 20 80 30

Largest:

80

Second largest:

50

Instead of sorting, maintain two variables.

```
int a[] = {10,50,20,80,30};
```

```
int largest = INT_MIN;
```

```
int second = INT_MIN;
```

```
for(int i=0; i<5; i++)
```

```
{
```

```
    if(a[i] > largest)
```

```
    {
```

```
        second = largest;
```

```
        largest = a[i];
```

```
    }
```

```
    else if(a[i] > second && a[i] != largest)
```

```
    {
```

```
        second = a[i];
```

```
    }
```

```
}
```

```
cout << second;
```

Output:

50

Include:

```
#include <climits>
```

Placement lesson

Don't sort when the problem can be solved in one traversal.

18. Frequency of Array Elements

Consider:

1 2 2 3 1 2

Frequency:

1 → 2

2 → 3

3 → 1

For a small known range, use a frequency array.

```
int a[] = {1,2,2,3,1,2};
```

```
int freq[10] = {0};
```

```
for(int i=0; i<6; i++)  
{  
    freq[a[i]]++;  
}
```

```
for(int i=0; i<10; i++)  
{  
    if(freq[i] > 0)  
        cout << i << " -> " << freq[i] << endl;  
}
```

Output:

1 -> 2

2 -> 3

3 -> 1

This is a very important bridge:

Array

↓

Frequency

↓

Hashing

19. Move All Zeros to the End

Input:

0 1 0 3 12

Expected:

1 3 12 0 0

Use a position pointer.

```
int a[] = {0,1,0,3,12};
```

```
int pos = 0;
```

```
for(int i=0; i<5; i++)
{
    if(a[i] != 0)
    {
        a[pos] = a[i];
        pos++;
    }
}
```

```
while(pos < 5)
{
    a[pos] = 0;
    pos++;
}
```

Output:

1 3 12 0 0

This introduces an important technique:

Write pointer / placement pointer

20. Array Rotation

Input:

1 2 3 4 5

Left rotation by one:

2 3 4 5 1

Simple method:

```
int first = a[0];
```

```
for(int i=0; i<4; i++)
{
    a[i] = a[i+1];
}
```

a[4] = first;

Result:

2 3 4 5 1

Later teach rotation by **K** positions using the **reversal algorithm**.

21. Two Sum

A classic placement problem.

Given:

2 7 11 15

Target:

9

We need:

2 + 7 = 9

Brute force

```
for(int i=0; i<n; i++)
{
    for(int j=i+1; j<n; j++)
    {
        if(a[i] + a[j] == target)
        {
            cout << a[i] << " " << a[j];
        }
    }
}
```

Complexity:

$O(n^2)$

Later teach the optimized hashing approach:

$O(n)$

This is exactly the progression students need for placements:

Brute Force → Optimization

22. Maximum Subarray Sum — Kadane's Algorithm

This deserves a complete classroom session.

Example:

-2 1 -3 4 -1 2 1 -5 4

Maximum subarray:

4 -1 2 1

Sum:

6

Core idea:

```
current = max(a[i], current + a[i]);  
best = max(best, current);
```

Implementation:

```
int a[] = {-2,1,-3,4,-1,2,1,-5,4};
```

```
int current = a[0];
```

```
int best = a[0];
```

```
for(int i=1; i<9; i++)
```

```
{
```

```
    current = max(a[i], current + a[i]);
```

```
    best = max(best, current);
```

```
}
```

```
cout << best;
```

Output:

6

Complexity:

$O(n)$

This is a **must-teach** placement algorithm.

PART B — 2D ARRAYS

23. What is a 2D Array?

A 2D array can be visualized as a table or matrix.

```
int a[3][3];
```

Conceptually:

```
    0  1  2
  +---+---+---+
0 |   |   |   |
  +---+---+---+
1 |   |   |   |
  +---+---+---+
2 |   |   |   |
  +---+---+---+
```

Access:

```
a[row][column]
```

Example:

```
a[1][2]
```

means:

row 1

column 2

24. Input and Output of Matrix

```
int a[2][3];
```

```
for(int i=0; i<2; i++)  
{  
    for(int j=0; j<3; j++)  
    {  
        cin >> a[i][j];  
    }  
}
```

Output:

```
for(int i=0; i<2; i++)  
{  
    for(int j=0; j<3; j++)  
    {  
        cout << a[i][j] << " ";  
    }  
  
    cout << endl;  
}
```

Nested loops are the key concept.

25. Row Sum

```
for(int i=0; i<3; i++)  
{  
    int sum = 0;  
  
    for(int j=0; j<3; j++)  
    {  
        sum += a[i][j];  
    }  
  
    cout << sum << endl;  
}
```

This demonstrates:

Outer loop → row

Inner loop → columns

26. Main Diagonal

For:

1 2 3
4 5 6
7 8 9

Main diagonal:

1 5 9

Condition:

$i == j$

Code:

```
for(int i=0; i<3; i++)  
{  
    cout << a[i][i] << " ";  
}
```

Output:

1 5 9

27. Secondary Diagonal

For:

1 2 3
4 5 6
7 8 9

Secondary diagonal:

3 5 7

Condition:

$i + j == n - 1$

Code:

```
for(int i=0; i<3; i++)  
{
```

```
    cout << a[i][2-i] << " ";  
}
```

Output:

3 5 7

This is an excellent MCQ topic.

28. Matrix Transpose

Matrix:

1 2 3
4 5 6

Transpose:

1 4
2 5
3 6

Concept:

$a[i][j] \rightarrow a[j][i]$

For a square matrix, an in-place transpose can be done:

```
for(int i=0; i<n; i++)  
{  
    for(int j=i+1; j<n; j++)  
    {  
        swap(a[i][j], a[j][i]);  
    }  
}
```

PART C — STRINGS IN C++

This section should be taught in **two stages**:

C-style strings

↓

std::string

That distinction is very important for MCQs.

29. Character vs String

A character:

```
char ch = 'A';
```

A string:

```
string name = "AMAR";
```

Notice:

'A' → character

"AMAR" → string

Single quotes:

'A'

Double quotes:

"AMAR"

This is a common MCQ trap.

30. C-Style String

```
char name[] = "AMAR";
```

Internally it contains:

A M A R \0

The last character is:

'\0'

called the **null character**.

It marks the end of a C-style string.

Therefore:

sizeof(name)

is:

5

not 4.

31. `strlen()` vs `sizeof()`

This is one of the **most important C++ string MCQ topics**.

```
char s[] = "HELLO";
```

```
cout << strlen(s) << endl;  
cout << sizeof(s);
```

Output:

5

6

Why?

`strlen()` counts characters before `'\0'`.

H E L L O

= 5

`sizeof()` counts the entire array:

H E L L O \0

= 6 bytes.

32. `std::string`

Modern C++ normally uses:

```
#include <string>
```

```
string name = "Amar";
```

Example:

```
#include <iostream>
#include <string>
using namespace std;

int main()
{
    string name = "Amar";

    cout << name;
}
```

Output:

Amar

33. String Indexing

```
string s = "HELLO";
```

```
cout << s[0];
cout << s[4];
```

Output:

H O

Indexes:

```
H E L L O
0 1 2 3 4
```

Just like arrays:

Strings are indexed from 0.

34. String Length

```
string s = "HELLO";
```

```
cout << s.length();
```

Output:

5

Also:

```
cout << s.size();
```

Output:

5

For `std::string`, `length()` and `size()` return the same size.

35. String Input

This is extremely important.

cin

```
string name;
```

```
cin >> name;
```

If input is:

Amar Panchal

only:

Amar

is read.

Why?

Because `>>` stops at whitespace.

36. `getline()`

To read an entire line:

```
string name;
```

```
getline(cin, name);
```

Input:

Amar Panchal

Output:

Amar Panchal

Common trap

```
int age;  
string name;
```

```
cin >> age;  
getline(cin, name);
```

The newline remaining after `cin >> age` may cause `getline()` to read an empty line.

Solution:

```
cin.ignore();  
getline(cin, name);
```

Or use an appropriate `ignore()` call when more robust handling is needed.

37. Traversing a String

```
string s = "HELLO";
```

```
for(int i=0; i<s.length(); i++)  
{  
    cout << s[i] << " ";  
}
```

Output:

H E L L O

Range-based:

```
for(char ch : s)  
{  
    cout << ch << " ";  
}
```

Same output.

38. Count Vowels

```
string s = "education";

int count = 0;

for(char ch : s)
{
    if(ch == 'a' || ch == 'e' || ch == 'i' ||
       ch == 'o' || ch == 'u')
    {
        count++;
    }
}

cout << count;
```

Output:

5

Students should learn to extend the logic for uppercase vowels as well.

39. Count Digits and Letters

Useful functions from `<cctype>`:

```
isalpha()
isdigit()
islower()
isupper()
isspace()
```

Example:

```
string s = "Amar123";

int letters = 0;
int digits = 0;
```

```
for(char ch : s)
{
    if(isalpha(ch))
        letters++;

    if(isdigit(ch))
        digits++;
}

cout << "Letters = " << letters << endl;
cout << "Digits = " << digits;
```

Output:

Letters = 4
Digits = 3

40. Convert Character Case

```
char ch = 'a';

cout << char(toupper(ch));
```

Output:

A

Similarly:

```
char ch = 'Z';

cout << char(tolower(ch));
```

Output:

z

Teach both:

```
toupper()
tolower()
```

and ASCII-based logic:

```
ch = ch - 'a' + 'A';
```

but make clear that `<cctype>` is usually cleaner.

41. Reverse a String

```
string s = "HELLO";
```

```
int i = 0;  
int j = s.length() - 1;
```

```
while(i < j)  
{  
    swap(s[i], s[j]);  
  
    i++;  
    j--;  
}
```

```
cout << s;
```

Output:

OLLEH

Again:

Two pointers

Same pattern as array reversal.

This is an important teaching moment:

Array reversal

↓

String reversal

↓

Same algorithmic pattern

42. Palindrome

A palindrome reads the same from both directions.

Examples:

MADAM
LEVEL

RADAR

Algorithm:

```
string s = "MADAM";

int left = 0;
int right = s.length() - 1;

bool palindrome = true;

while(left < right)
{
    if(s[left] != s[right])
    {
        palindrome = false;
        break;
    }

    left++;
    right--;
}

if(palindrome)
    cout << "Palindrome";
else
    cout << "Not Palindrome";
```

Output:

Palindrome

This is a **must-know placement program**.

43. Character Frequency

Suppose:

programming

We want to count every character.

Use:

```
int freq[256] = {0};
```

Then:

```
for(char ch : s)
{
    freq[(unsigned char)ch]++;
}
```

For simple ASCII input, students can conceptually understand it as:

character $\rightarrow$ index $\rightarrow$ frequency

This pattern is extremely important.

44. First Non-Repeating Character

Example:

swiss

Frequency:

s $\rightarrow$ 3

w $\rightarrow$ 1

i $\rightarrow$ 1

First character whose frequency is 1:

w

Code:

```
string s = "swiss";
```

```
int freq[256] = {0};
```

```
for(char ch : s)
{
    freq[(unsigned char)ch]++;
}
```

```
for(char ch : s)
{
    if(freq[(unsigned char)ch] == 1)
    {
        cout << ch;
        break;
    }
}
```

```
}  
}
```

Output:

w

This teaches an excellent interview pattern:

First pass → collect information

Second pass → make decision

45. Anagram

Two strings are anagrams if they contain the same characters with the same frequencies.

Example:

listen
silent

Output:

Anagram

A frequency-based solution:

```
int freq[256] = {0};
```

```
for(char ch : s1)  
    freq[(unsigned char)ch]++;
```

```
for(char ch : s2)  
    freq[(unsigned char)ch]--;
```

```
bool anagram = true;
```

```
for(int i=0; i<256; i++)  
{  
    if(freq[i] != 0)  
    {  
        anagram = false;  
        break;  
    }  
}
```

This is better to teach than simply sorting because it develops the **frequency-counting technique**.

46. `find()`

```
string s = "I love programming";
```

```
cout << s.find("programming");
```

Output:

7

Indexing:

```
I _ l o v e _ p...  
0 1 2 3 4 5 6 7
```

If the substring is not found:

`string::npos`

is returned.

Example:

```
if(s.find("Java") == string::npos)  
{  
    cout << "Not Found";  
}
```

47. `substr()`

```
string s = "Programming";
```

```
cout << s.substr(0, 7);
```

Output:

Program

Syntax:

```
s.substr(start, length);
```

Another example:

```
cout << s.substr(3, 4);
```

Output:

gram

Students must understand:

Second argument is **length**, not ending index.

48. **erase()**

```
string s = "Hello World";
```

```
s.erase(5, 1);
```

```
cout << s;
```

Output:

HelloWorld

Syntax:

```
s.erase(start, count);
```

49. **insert()**

```
string s = "HelloWorld";
```

```
s.insert(5, " ");
```

```
cout << s;
```

Output:

Hello World

50. String Comparison

```
string s1 = "apple";
string s2 = "banana";

if(s1 < s2)
    cout << "s1 comes first";
```

Output:

s1 comes first

String comparison is generally **lexicographical**.

Students should understand that comparison is not based on string length alone.

51. Remove Duplicate Characters

Example:

programming

Expected concept:

progamin

Use a frequency/visited structure.

```
string s = "programming";
bool seen[256] = {false};

string result = "";

for(char ch : s)
{
    unsigned char u = static_cast<unsigned char>(ch);

    if(!seen[u])
    {
        result += ch;
        seen[u] = true;
    }
}

cout << result;
```

The exact output for this input is:

52. Reverse Words in a Sentence

Input:

I love programming

Output:

programming love I

This is more advanced because students must understand:

characters

↓

words

↓

sentence

It is an excellent transition from basic strings to interview problems.

53. String Compression

Input:

aaabbccccc

Output:

a3b2c4

Core idea:

Current character

+

Count consecutive occurrences

This introduces **run-length encoding** and is a good medium-level placement question.

54. Longest Substring Without Repeating Characters

This is a very important placement problem.

Input:

abcabcbb

Longest substring without repeating characters:

abc

Length:

3

This introduces:

Sliding Window

Students should learn this only after they understand:

String traversal

+

Frequency

+

Two pointers

Otherwise it becomes "memorise this magic code"—and magic code is excellent for exams and terrible for interviews.

PART D — ARRAY + STRING MCQ MASTER AREA

For MCQs, I would make students particularly strong in these areas.

Arrays

Question 1

```
int a[] = {10,20,30,40};
```

```
cout << a[2];
```

Output:

30

Question 2

```
int a[5] = {1,2};
```

```
for(int i=0;i<5;i++)  
    cout << a[i] << " ";
```

Output:

1 2 0 0 0

Question 3

```
int a[] = {10,20,30};
```

```
cout << sizeof(a);
```

Assuming `int` is 4 bytes:

12

Question 4

```
int a[] = {10,20,30};
```

```
cout << sizeof(a)/sizeof(a[0]);
```

Output:

3

Question 5

Understand:

`a[i]`

and:

`*(a+i)`

represent equivalent element access in the usual expression context.

String MCQ Traps

Question 6

```
char s[] = "HELLO";
```

```
cout << strlen(s);  
cout << sizeof(s);
```

Output:

56

if printed consecutively without spacing.

Conceptually:

```
strlen = 5  
sizeof = 6
```

Question 7

Difference:

'A'

and:

"A"

Answer:

'A' → character literal

"A" → string literal

Question 8

```
string s = "HELLO";
```

```
cout << s[1];
```

Output:

E

Question 9

```
string s = "HELLO";
```

```
cout << s.length();
```

Output:

5